

COFFE journal study: study guide

Paper: On the Time Efficiency of Code Generated and Optimized by Large Language Models. Yun Peng, Jun Wan, Xiaoxue Ren, Tim Menzies, and Michael R. Lyu. IEEE Transactions on Software Engineering, 2026. DOI: 10.1109/TSE.2026.3709872.

Prepared 6 October 2026 from the downloaded 23-page accepted author manuscript. Page references use that PDF. These are study notes and a critical assessment, not a replication. The organization follows Keshav's three-pass method [2].

Pass 1: the central idea

Correctness tests are often too small to expose performance differences. COFFE adds stressful inputs, mutates their scale, and evaluates code with CPU instruction counts and a combined correctness-efficiency metric. It studies Python code generation and seven optimization approaches. The main contribution is an evaluation framework and empirical evidence, not a new general-purpose code optimizer. [1, pp. 1-4, 9]

The five Cs

Category: Benchmark construction, test generation/mutation methods, metric design, and empirical evaluation. **Context:** Correct code can still be slow; improvements can depend on workload scale; elapsed-time measurements can fluctuate. [1, pp. 1-3]

Correctness: Existing tests, generated input contracts, target-program execution, and an LLM conflict checker support validation. They do not provide a complete behavioral specification. Instruction count is a performance proxy. [1, pp. 5-9, 19]

Contributions: COFFE; STGEN for stressful tests; STMUT for varied input scales; efficient@k; experiments across reported model families and optimization approaches. [1, pp. 2-3]

Clarity - reviewer assessment: The five research questions give a useful structure. Care is needed because the paper uses RSD both for timing stability and variation across workloads, and uses different denominators for efficiency and speedup.

This is the journal extension

The earlier FSE 2025 COFFE paper is not interchangeable with this article. The journal version adds input-scale mutation, evaluation of robustness across scales, and comparison of seven code-optimization approaches. Those additions are directly relevant to our proposed PhD study. [1, p. 3]

Essential terms

Stressful test: A large/expensive valid input intended to reveal efficiency differences. **Contract:** Assertions describing permitted input types, bounds, and relationships. **STGEN:** Contract-guided generation of stressful inputs. **STMUT:** Mutation toward smaller, distinct input-cost levels. **efficient@k:** Estimated probability of obtaining at least one test-correct candidate that beats the benchmark reference within k samples. [1, pp. 4-9]

The benchmark covers individual functions and standalone program files. It deliberately excludes repository-level tasks, whose dependencies and interactions complicate measurement. [1, p. 3]

Pass 2: benchmark and test-generation pipeline

Data construction

Split	Sources	Tasks
Function-level	HumanEval 164 + sanitized MBPP subset 234	398
File-level	Code Contests 58 + APPS 300	358
Three-scale subset	Function tasks with three valid mutated levels	307
Three-scale subset	File tasks with three valid mutated levels	212

Sources: [1, pp. 4, 8, Tables I-III]. Validate reference solutions against original tests, remove unsupported/inconsistent cases, and select file-level problems that models can solve. The file split is deliberately filtered toward solvability. On average, a function task has one reference solution; a file task has 66.93. That difference affects how hard it is to beat the best reference. [1, pp. 4, 8]

STGEN: generate stressful tests

Phase 1 - contracts: Insert assertions describing valid input types, scales, and dependencies. For functions, assertions check parameters; for files, they are placed around input statements or after input-reading loops. Check each added assertion against existing correctness tests and regenerate failed assertions. [1, pp. 5-6]

Phase 2 - inputs: Generate compact expressions or generator functions instead of writing enormous input literals. Execute the input-generating code, then check the resulting input with the contracts and reference program. Reuse verified tests as examples. [1, p. 6]

Phase 3 - resolve conflicts: After repeated assertion failures, use an LLM judge to decide whether a contract or test is wrong, then regenerate the relevant item. The stated trigger is five conflicts. Once judged valid, a contract is not rechecked by this stage. The judge is a fallible part of the pipeline, not an independent proof. [1, p. 6]

STMUT: vary input scale

A description-based mutator targets an intermediate scale between a small correctness input and a stressful input. A statement-based mutator profiles both inputs, identifies a statement whose share of execution time changes most, and asks for an input reducing its executions or per-execution cost. Repeat with depth three. Filter/group mutated tests using differences in CPU instruction cost, and retain three levels where possible. L1 is largest; L3 is smallest. [1, pp. 6-8]

Select and measure tests

Generate 20 stressful tests per problem. Measure each 12 times, drop the highest and lowest results, and average the remaining ten. Use a five-second execution cap per measurement, then retain the five most costly tests. Keep original correctness tests as well. The scale experiment includes only tasks for which three levels can be obtained; it is a smaller evaluation population. [1, pp. 8-9]

Pass 2: metrics, comparators, and evidence

Read the metrics precisely

CPU instruction count: Count executed CPU instructions with Cirron/Linux facilities. This is distinct from Python bytecode count and elapsed time. **Speedup:** The method defines the ratio of reference to generated-solution instruction counts. Reported speedup tables average over correct outputs; efficient@1 and pass@1 use all task instances. [1, pp. 8-10, 13, 16]

efficient@k: For each problem, generate n candidates and count c_f candidates that both pass tests and beat the best reference. Estimate $1 - \text{choose}(n - c_f, k) / \text{choose}(n, k)$, then average across problems. Replacing c_f by the count of correct candidates gives pass@k. For $k=1$, the per-problem expression is c_f/n . Beating a reference is not automatically the same as improving the particular input program. [1, p. 8; final distinction is reviewer interpretation]

RSD: Standard deviation / mean. Low RSD across repeated measurements indicates stability. High RSD across different solutions indicates test distinguishability. Low RSD of efficient@1 across three input levels is the paper's aggregate robustness measure. These are three different uses of the same statistic. [1, pp. 9, 11, 14]

Experiments and comparators

The paper reports 19 model families, with several size variants, and seven optimization approaches: CoT, in-context learning, SBLLM, PIE, Self-refine, EffiLearner, and DeepSeek R1. Prompt-engineering baselines use GPT-4o. Optimization targets include GPT-4o outputs and reference solutions. Measurements run on an Ubuntu server; tables report five experimental repetitions and 95% confidence intervals for efficiency/speedup. [1, pp. 9-10, 12, 15]

RQ1-RQ3: validity of the evaluation machinery

Question	Author-reported result
RQ1: measurement stability	Instruction-count RSD: 0.003%-0.005%; runtime RSD: 2.37%-13.27%. Pearson correlation: 0.90-1.00.
RQ2: STGEN input quality	Accuracy: 98.64% function / 98.91% file. Line coverage: 96.01% / 95.17%.
RQ2: distinguishability	For DeepSeek V3 outputs, RSD across solutions rises from 23.02% to 39.11% (function), and 18.07% to 30.67% (file).
RQ3: STMUT scale diversity	Diff%: 48.59% function / 27.16% file, versus few-shot baselines 39.05% / 26.74%.

Sources: [1, pp. 10-12, Tables V-VII]. Test-generation accuracy is operationally defined by successful target execution; it is not an independent proof that every generated input respects the intended specification. STMUT's gain over few-shot mutation is much smaller for files than for functions.

Pass 2: the ten findings, condensed

These are findings under the paper's benchmark, model versions, and metrics. They are not universal statements about every LLM or every workload.

- 1. Leaders depend on the metric.** DeepSeek R1 has the highest function-level efficient@1 (54.17%); Llama3.1 has the highest file-level value (42.71%). A different model can lead on speedup among correct outputs. [1, pp. 12-13]
- 2. Correctness exceeds efficiency.** The best pass@1 values are 81.66% and 90.78% for function/file tasks, above the best efficient@1 values. These maxima may belong to different models; their difference is not a paired error rate for a single model. [1, pp. 12-13; denominator qualification added here]
- 3. Files are harder to make efficient.** File-level outputs generally have lower reference-relative efficiency and larger gaps between pass@1 and efficient@1. [1, p. 13]
- 4. Size is not sufficient.** Larger models generally improve correctness, but additional parameters do not consistently improve efficient@1. This is an observational comparison, not a controlled experiment isolating parameter count. [1, pp. 13-14]
- 5. Input scale matters more for files.** Function-level efficient@1 varies less across the three mutated levels; file-level variation can be much larger. [1, pp. 14-15]
- 6. Fastest and most stable need not coincide.** In particular, a model with strong file-level efficiency may still vary substantially across scales. [1, p. 15]
- 7. Optimization methods help, unevenly.** EffiLearner has the highest efficient@1 on three of four target/scope combinations; DeepSeek R1 leads on GPT-4o function outputs. EffiLearner reaches 59.08% on GPT-4o file outputs and 54.72% on function references. The paper's "about half" summary concerns its benchmark metric, not an audited fraction of deployment-safe edits. [1, pp. 15-16, Table X]
- 8. Correctness and efficiency both limit optimization.** The paper attributes more function-level failures to correctness, and more file-level failures to insufficient efficiency, using its deficit ratio. The starting-program qualification on the next page is important. [1, p. 16]
- 9. Optimization does not remove scale sensitivity.** Many approaches remain less stable on file tasks, and the initial program can influence the optimized program's robustness. [1, pp. 16-17]
- 10. No method leads every objective.** The strongest efficient@1 and lowest across-scale RSD do not consistently belong to the same optimizer. This is evidence of different rankings, not proof that efficiency and robustness cannot be improved together. [1, p. 17]

Leakage analysis and implications

The authors compare against four accessible training datasets using text overlap and exact normalized-AST matching. They remove 33 overlapping function problems and re-evaluate; they also report 232 verbatim, reference-beating outputs across the inspected function-level model outputs. They find no exact matches in the inspected file split. These checks cover the examined training sets and matching rules, not all possible training exposure. [1, pp. 17-19]

The practical implication is to evaluate correctness, reference-relative efficiency, and workload sensitivity together. The conclusion contains a count of 11 findings, but the results section numbers ten; this guide follows the ten numbered findings. [1, pp. 12-17, 20]

Pass 3: critical appraisal and the PhD connection

Strengths

The paper provides a concrete pipeline for obtaining large valid inputs, handles input dependencies, and tests both functions and files. It examines measurement noise, workload scales, code generation, optimization, and some training-data overlap. These make it a strong baseline for a study of optimization judgments. [1, Sections III-VIII; reviewer assessment]

Limitations and interpretation checks

Instruction count is not elapsed runtime: Different instructions, memory behavior, and code paths can have different costs. High overall correlation does not guarantee that every pair has the same ranking on instruction count and latency. The authors acknowledge imperfect representativeness. A follow-up should validate both metrics where the deployment objective is time. [1, pp. 8, 11, 19; reviewer implication]

Robustness is aggregated: Low variation in overall efficient@1 across levels can conceal individual programs that improve at one level and regress at another. The number of successful tasks can stay stable while the identity of successful tasks changes. Candidate-level worst-case regressions require a different analysis. [1, pp. 14-17; reviewer inference]

Reference and selection effects: File tasks have many more reference solutions, and are filtered using model solvability. Comparing function/file metrics therefore mixes program scope, task difficulty, and reference strength. The three-level subset further excludes tasks not successfully mutated. [1, pp. 4, 8-9]

Correctness deficit is not necessarily introduced damage: Table X uses 100% - pass@1 as a correctness deficit. For an initially correct reference this is informative; for GPT-4o inputs that may already be wrong, it is not the fraction of newly broken programs. A paired before/after transition table is needed to identify correct-to-incorrect edits and repairs. [1, pp. 13, 15-16; reviewer analysis]

Metric interpretation: efficient@1 asks whether outputs beat a benchmark reference. It does not directly measure whether an acceptance policy rejects harmful candidates, retains useful ones, or abstains when evidence is weak. Speedup calculated only on correct outputs also has a different denominator. [1, pp. 8-9, 13, 16]

Validity and reproducibility: Generated contracts and the LLM judge can share blind spots. Open training-set checks cannot certify absence of leakage. Before reproducing the Python PIE baseline, identify its actual checkpoint/version and adaptation: the separate ICLR PIE PDF we downloaded studies C++. Also record model/iteration budgets and available optimizer feedback. [1, pp. 6, 9-10, 17-19; replication questions]

What is already done, and what remains a question

Already done here: stressful performance tests, generated contracts, input-scale mutation, repeated measurements, an efficiency-plus-correctness metric, and robustness comparisons. A study that merely adds different input sizes would overlap directly.

A narrower proposed study would audit individual acceptance decisions across iterative Python edits, using held-out valid workloads and independent measurements. Measure false approvals, useful edits rejected, uncertainty/deferrals, and execution cost. Compare with COFFE-style multi-scale evaluation and SemOpt's existing regression rule. Whether a new method is needed depends on that evidence; no novel gap is established by these notes alone.

Recall, reproduction, and source map

Questions to answer without the paper

1. Why are ordinary unit tests insufficient for performance evaluation? 2. How do contracts help STGEN, and how can they be wrong? 3. Why generate expressions/functions rather than long literals? 4. What extra role does STMUT play? 5. What does efficient@1 require? 6. Why is instruction-count stability different from runtime validity? 7. Why can stable aggregate scores conceal individual regressions? 8. Why is the journal extension essential to our LR?

Minimum reproduction record

Preserve the exact selected tasks, reference solutions, original tests, generated contracts, input generators, mutation seeds, accepted/rejected inputs, and level assignments. Record model versions, optimizer checkpoints, prompts, iteration budgets, interpreter and dependency versions, container setup, hardware counters, timeouts, and repeat/warm-up rules. The public COFFE repository was originally released with the conference paper; verify that downloaded artifacts actually contain the journal's STMUT and optimizer experiments before claiming a journal replication. [1, pp. 3-10, 19; replication checklist]

Where to return in the original

Need	Source pages
Motivation and journal additions	1-3
Task selection and benchmark populations	3-4, 8-9; Tables I-III
STGEN contracts, generators, conflict checking	4-6; Figure 2
STMUT profiling and input levels	6-8; Figure 3
Instruction counts and efficient@k	7-9; Equations 1-3
Models, comparators, RQ1-RQ3	9-12; Tables IV-VII
Generation and optimization findings	12-17; Tables VIII-XI
Implications and leakage analysis	17-19; Tables XII-XIV
Threats, related work, conclusion	19-20

References and accessible materials

[1] Y. Peng, J. Wan, X. Ren, T. Menzies, and M. R. Lyu. On the Time Efficiency of Code Generated and Optimized by Large Language Models. IEEE TSE, 2026, accepted author version, 23 pages. DOI: <https://doi.org/10.1109/TSE.2026.3709872>

Author PDF used: <https://timm.fyi/26coffe.pdf>

[2] S. Keshav. How to Read a Paper. ACM SIGCOMM Computer Communication Review 37(3), 83-84, 2007; author revision, 17 February 2016. Reading framework only. <https://svr-sk818-web.cl.cam.ac.uk/keshav/papers/07/paper-reading.pdf>

[3] COFFE public benchmark repository. Repository contents/version were not executed or fully audited for this guide. <https://github.com/JohnnyPeng18/Coffe>

Suggested presentation message: evaluating optimization requires appropriate workloads and clearly defined metrics; reproducible measurements and aggregate scores still leave candidate-level acceptance decisions to be studied.